

Aeternum-core Smart Contract Audit

Internal Security Audit Report — Revision 3

Contract:	AeternumVault.sol
Repository:	github.com/Aeternumlabs/aeternum-core
Prepared By:	Ndubuisi Ugwuja — Founder, Aeternum Labs
Date:	7th July 2026
Audit Type:	Internal Pre-Audit Security Assessment
Status:	Complete — External audit pending pre-mainnet

REVISION NOTE:

This document supersedes the internal audit dated 3rd June 2026 (Revision 2). The contract has been migrated from Chainlink Automation (*checkUpkeep()* / *performUpkeep()*, forwarder-gated) to a permissionless keeper architecture (*getTriggerableVaultsBatch()* / *triggerRecovery()*). The on-chain scan cursor and the *setForwarder()* access-control surface have been removed entirely — recovery execution is now callable by any address. All findings, test results, coverage figures, gas costs, and static-analysis results in this revision reflect the current contract state following that migration. Underlying test evidence was generated 25th June 2026.

Table of Contents

1. Executive Summary
2. Project Goals
3. Contract Architecture and Trust Model
4. Automated Testing
5. Static Analysis — Slither
6. Linting — Solhint
7. Detailed Findings
8. Codebase Maturity Evaluation
9. Known Limitations and Out of Scope
10. Tool Versions

1. Executive Summary

Engagement Overview

AeternumVault is a trustless, non-custodial smart wallet vault with built-in automated ETH recovery. Users deposit ETH into a self-sovereign vault, configure a backup address and inactivity period, and if they ever go permanently inactive — lost keys, death, incapacitation — their ETH is transferred to the backup address automatically. No custodians. No admin backdoors. No subscription fees. Just code.

This revision reflects the migration away from Chainlink Automation to a permissionless keeper architecture. *checkUpkeep()* and *performUpkeep()* — along with the on-chain scan cursor and the *setForwarder()-gated* caller restriction — have been removed. Recovery is now driven by three functions available to any external caller:

- *getTriggerableVaultsBatch(uint256 startIndex, uint256 batchSize)* — a stateless, paginated view function that identifies wallets eligible for recovery within a given index window.
- *isRecoveryDue(address wallet)* — a single-wallet view function returning whether a given wallet is currently eligible for recovery.
- *triggerRecovery(address wallet)* — a permissionless execute function that any address may call to complete recovery for a single due wallet. It independently re-validates all preconditions regardless of who calls it or what off-chain data motivated the call.

The Aeternum Labs keeper bot (Phase 1 of the rollout) performs scanning off-chain using a two-layer filter — a database pre-filter followed by a batched on-chain call to *isRecoveryDue()* for validation — before submitting *triggerRecovery* transactions, typically batched via Multicall3 for gas efficiency. Gelato is planned as an independent backup keeper in Phase 2 (Hybrid Account), and Chainlink CRE is planned to join as a multi-chain orchestrator in Phase 3 (Multichain) — a single off-chain service running independent, per-chain recovery workflows, with no cross-chain messaging of any kind. Because *triggerRecovery* is fully permissionless, no single automation provider is a point of failure for fund recovery.

The review employed:

- Static analysis using Slither v0.11.4
- Manual line-by-line code review
- 127-test Foundry suite (0 failing, 0 skipped) achieving 100% line and function coverage on AeternumVault.sol
- Foundry fuzz (6 functions) and invariant tests (3 properties, 128,000+ calls each)
- Echidna property-based fuzzing across 12 invariant properties with 50,127 randomised call sequences

- Linting with Solhint

No High or Medium severity vulnerabilities remain open. The one Medium finding from Revision 2 (external calls inside the *performUpkeep* loop) is resolved as a direct consequence of the architecture migration — see Section 7.

Finding Summary

Severity	Count	Status
High	0	—
Medium	0	1 finding resolved in this revision — see AV-02
Low	2	Acknowledged (mitigated / acceptable)
Informational	5	Acknowledged

Resolved since Revision 2: 3 findings (1 Medium, 2 Informational) — see Section 7 for detail.

2. Project Goals

This assessment was scoped to provide a thorough internal security review of the post-migration AeternumVault contract. The following questions were answered:

- Does the vault correctly enforce user-specific balance isolation, preventing any cross-user fund access?
- Are all ETH-transferring code paths protected by the Checks-Effects-Interactions (CEI) pattern and ReentrancyGuard?
- Is the permissionless keeper architecture safe — does *isRecoveryDue* accurately reflect on-chain due-wallet status for the keeper bot's pre-submission validation pass, and does *triggerRecovery* independently re-validate each wallet before acting, regardless of which address submits the transaction or how many wallets are batched into a single transaction?
- Does removing the forwarder-gated automation model introduce any new attack surface now that *triggerRecovery* is callable by any address?
- Do multiple keepers (or backup automation providers) racing to trigger the same wallet resolve safely, without reverting or double-transferring funds?

- Does *getTriggerableVaultsBatch* paginate the registry correctly and never revert on edge-case inputs (empty registry, out-of-range start index, zero batch size)?
- Does the swap-and-pop registry removal maintain array integrity under sequential recoveries?
- Does the abandonment mechanism after MAX_RECOVERY_ATTEMPTS prevent infinite retry loops that waste keeper gas?
- Are all access control paths — the *onlyRegistered* modifier — enforced correctly now that no forwarder restriction exists?
- Does the contract correctly reject direct ETH transfers to prevent accidental fund lock?
- Do all 12 Echidna invariant properties hold across 50,000+ randomised call sequences?
- Are there any High or Medium severity static analysis findings not previously addressed?

3. Contract Architecture and Trust Model

AeternumVault remains a single-contract architecture. There is no subscription model, no treasury, and no fee collection. Each registered wallet has its own isolated vault — balances are tracked individually per user via a private mapping, never pooled.

Trust Model

Actor	Can Do	Cannot Do
User	Register, deposit, send, withdraw all, ping, update config, cancel recovery	Access other users' funds
Any external caller (Aeternum Labs keeper bot, a future backup automation provider, or an individual)	Call <i>triggerRecovery(address)</i> to execute recovery once a wallet's inactivity period has elapsed; call <i>getTriggerableVaultsBatch</i> or <i>isRecoveryDue</i> as read-only views	Alter user configs, redirect funds to any address other than the pre-registered backup address, or trigger recovery before the inactivity period elapses

Actor	Can Do	Cannot Do
No one	—	Pause recovery, upgrade the contract, collect fees, or access user funds

Key Immutables (Set at Deployment)

Immutable	Description and Recommended Value
MIN_INACTIVITY_PERIOD	Minimum inactivity period in seconds. Enforced on <i>register()</i> and <i>updateInactivityPeriod()</i>
MAX_INACTIVITY_PERIOD	Hard ceiling on inactivity period, preventing permanent lock-up. Recommended: 10 years (3650 days)
MAX_RECOVERY_ATTEMPTS	Consecutive failed recovery attempts before a vault is permanently abandoned. Recommended: 3

Note: MAX_CHECK_UPKEEP_SIZE, MAX_PERFORM_UPKEEP_SIZE, and CURSOR_ADVANCE_INTERVAL from the Chainlink Automation architecture have been removed. *getTriggerableVaultsBatch* now takes an explicit batch window (*startIndex*, *batchSize*) as call-time parameters rather than reading fixed on-chain limits, and scan state lives entirely off-chain in the keeper bot rather than in contract storage.

RecoveryConfig Struct

Fields: *backupAddress*, *inactivityPeriod*, *lastActivity*, *balance*, *isActive*, *failedRecoveryAttempts*, *isAbandoned*. Unchanged from Revision 2.

Core Components

User Vault (per address)

Each user registers with a backup address and inactivity period. Deposited ETH is held under their address key in the contract. The *onlyRegistered* modifier admits both *isActive* and

isAbandoned wallets, ensuring abandoned wallet owners retain full access to their remaining balance via *withdrawAll()* or *send()*.

Permissionless Keeper Architecture

Off-Chain Scan, On-Chain Validation *getTriggerableVaultsBatch(uint256 startIndex, uint256 batchSize)* is a public view function returning the subset of registered wallets — within the given index window — whose inactivity period has elapsed and which hold a non-zero balance. It performs no state changes and can be called off-chain at zero gas cost via *eth_call*, or on-chain by any caller who wants due-wallet data sourced directly from the contract rather than an indexer. Unlike the prior *checkUpkeep()* cursor, there is no persistent on-chain scan position — each call is stateless and explicitly parameterised by the caller. This function is not part of the Aeternum Labs keeper bot's live validation path — see *isRecoveryDue* below and Section 4.5 for the gas-cost distinction.

isRecoveryDue(address wallet) is a public view function returning a single boolean — true if the given wallet is currently registered, active, holds a non-zero balance, and its inactivity period has elapsed. It is cheaper and simpler than *getTriggerableVaultsBatch* (a single boolean comparison, no array construction or pagination), which makes it well suited to batching many per-wallet checks into a single *eth_call* via multicall. This is the function the Aeternum Labs keeper bot actually calls — once per database pre-filter candidate, batched into a single on-chain multicall — immediately before submitting *triggerRecovery*, to catch any wallet whose status changed since the database was last synced.

triggerRecovery(address wallet) is a public, permissionless function that executes recovery for a single wallet if and only if the wallet is registered, active, holds a non-zero balance, and its inactivity period has elapsed at the moment of the call. It re-validates all preconditions independently of any off-chain data, silently returning — no revert, no state change — if the wallet is not actually due, protecting callers acting on stale or duplicate data from wasted gas beyond the ~8,288 gas floor cost of the precondition check. Any address may call it, including the wallet owner, the designated backup address, or an unrelated third party.

The Aeternum Labs keeper bot (Phase 1) implements a two-layer scan: a database pre-filter (indexed from on-chain events by the Ponder-based indexer) narrows the registry to likely-due wallets in batches of up to `KEEPER_BATCH_SIZE`, polling roughly every `KEEPER_POLL_INTERVAL_MS`; candidates are then re-validated via a single batched on-chain multicall call to *isRecoveryDue* for each one, immediately before *triggerRecovery* is submitted, eliminating false positives from indexer lag. As the keeper bot's own source documents it, this two-layer scan is "an efficiency layer, not a security boundary" — if either the database or the validation pass ever drifts from on-chain truth, *_executeRecovery*'s own precondition checks remain the final arbiter of whether any ETH actually moves. Gelato is planned as an

independent backup keeper in Phase 2 (Hybrid Account); Chainlink CRE is planned to join in Phase 3 (Multichain) as a multi-chain orchestrator — a single off-chain service operating one independent workflow per chain, each scanning and submitting *triggerRecovery* transactions to that chain's own deployment, with no messaging or dependency between chains. All three keepers run independently against the same permissionless entry point, with no contract change required to add any of them.

Batched Submission via Multicall3

Confirmed-due wallets are not submitted as one *triggerRecovery* transaction per wallet. The keeper bot batches multiple calls into a single transaction via Multicall3's `aggregate3(calls, allowFailure: true)` function, up to the hard-coded `MAX_CALLS_PER_TX` ceiling of 120 calls per transaction. `allowFailure: true` ensures one wallet's failed ETH transfer within a batch — e.g. a backup address that rejects ETH — does not cause any other wallet's recovery in the same transaction to revert, complementing the contract's own per-call silent-return isolation described above. Multicall3 (0xcA11bde05977b3631167028862bE2a173976CA11) is a widely-deployed, immutable, canonical multicall contract used broadly across the Ethereum ecosystem. It is a dependency of the keeper bot's transaction-submission layer, not of AeternumVault.sol itself, and is outside the scope of this contract-specific audit — see Section 9.

Race Safety: Because *triggerRecovery* is permissionless and idempotent-by-precondition, multiple keepers (or a keeper racing an individual actor) resolve safely — the first successful call executes recovery and removes the wallet from the registry; any subsequent call silently returns without reverting or double-transferring funds

(*test_triggerRecovery_raceCondition_secondCallerSilentlyReturns*, *test_staleData_independentExecution_whenOneBotRaceLost*). This holds whether the competing calls arrive in separate transactions or are batched together via Multicall3 within the same transaction: the EVM executes every call in a transaction sequentially, so there is no concurrent storage access for this safety property to race against.

Registry

s_registeredWallets(address[]) tracks all active vaults. *s_walletIndexPlusOne* provides $O(1)$ position lookup via a 1-indexed mapping, enabling safe swap-and-pop removal without corrupting array integrity across sequential recoveries — including sequences of removals batched into a single transaction via Multicall3.

Abandonment Mechanism

After `MAX_RECOVERY_ATTEMPTS` consecutive failed ETH transfers to a backup address, the vault is permanently marked *isAbandoned* = true and removed from the monitoring registry.

The user retains full access to their balance via *withdrawAll()* or *send()*. Re-registration using the same backup address is permanently blocked via the *s_abandonedBackupAddresses* mapping.

4. Automated Testing

4.1 Unit Test Coverage (Foundry)

Metric	Result
Line Coverage (AeternumVault.sol)	100.00% (167/167)
Statement Coverage	99.51% (203/204)
Branch Coverage	97.14% (34/35)
Function Coverage	100.00% (22/22)
Total Tests	127 passing, 0 failing, 0 skipped

Coverage figures above are for AeternumVault.sol. Deployment scripts (Deploy.s.sol, HelperConfig.s.sol) and the Echidna harness (AeternumVaultEchidna.sol) show 0% coverage as expected — scripts are tested by live execution, not unit tests. Mock contracts show partial coverage reflecting only the paths exercised during security testing.

Test sections covered: Constructor and deployment validation, Registration (standard and re-registration after abandonment), Deposit, Send, Withdraw all, Ping, Config updates (backup address, inactivity period), Cancel recovery, *getTriggerableVaultsBatch* (pagination, clamping, empty registry, zero batch size, mixed due/not-due results), *triggerRecovery* (execution, permissionless caller checks, stale-data silent returns, race conditions, failure and retry, abandonment at max attempts), keeper bot full-cycle integration tests, Registry integrity (swap-and-pop batch removal), Reentrancy defense, view helpers, fuzz tests, invariant tests.

4.2 Fuzz Tests

Six fuzz test functions were run with 256 runs each:

- *testFuzz_getTriggerableVaultsBatch_neverReverts(uint256,uint256)* — no input combination of *startIndex*/*batchSize* causes a revert

- *testFuzz_deposit_amount(uint128)* — any non-zero deposit amount handled correctly
- *testFuzz_isRecoveryDue_timing(uint256)* — recovery only triggers at or after the inactivity period
- *testFuzz_register_validInactivityPeriod(uint256)* — any period within [MIN_INACTIVITY_PERIOD, MAX_INACTIVITY_PERIOD] accepted
- *testFuzz_send_amount(uint128)* — partial sends preserve balance invariants
- *testFuzz_triggerRecovery_neverRevertsForArbitraryAddress(address)* — *triggerRecovery* never reverts regardless of caller

All passed.

4.3 Foundry Invariant Tests

ID	Description	Calls	Result
FUND-INV-1	Contract ETH balance >= sum of all user vault balances (<i>invariant_contractBalanceCoverAllUserFunds</i>)	128,000	PASS
FUND-INV-2	Every wallet in the registry has <i>isActive</i> == true (<i>invariant_registeredWalletsAreAlwaysActive</i>)	128,000	PASS
FUND-INV-3	Abandoned wallets are never present in the registry (<i>invariant_abandonedWalletsNotInRegistry</i>)	128,000	PASS

Each invariant ran 256 sequences totalling 128,000 calls, with the large majority of calls reverting on precondition guards (expected and healthy behaviour for invariant fuzzing — it confirms guard conditions are actually being exercised rather than trivially satisfied).

4.4 Echidna Property-Based Fuzzing

Configuration

testMode: property, testLimit: 50,000, seqLen: 100, shrinkLimit: 5,000. Result: 50,127 calls executed (slightly over testLimit, as expected). All 12 properties passing. Unique instructions: 2,928. Corpus size: 8. Seed: 2427907532070541605.

ID	Property	Result
PROP-01	<i>echidna_zeroBalanceNeverRecoveryDue</i>	PASS
PROP-02	<i>echidna_registeredWalletsAreAlwaysActive</i>	PASS
PROP-03	<i>echidna_contractBalanceCoversUserFunds</i>	PASS
PROP-04	<i>echidna_walletPeriodAboveMin</i>	PASS
PROP-05	<i>echidna_inactivityPeriodNeverExceedsMax</i>	PASS
PROP-06	<i>echidna_activeAndAbandonedAreMutuallyExclusive</i>	PASS
PROP-07	<i>echidna_getTriggerableVaultsBatchConsistency</i>	PASS
PROP-08	<i>echidna_abandonedWalletsNotInRegistry</i>	PASS
PROP-09	<i>echidna_failedAttemptsNeverExceedMax</i>	PASS
PROP-10	<i>echidna_registryLengthIsConsistent</i>	PASS
PROP-11	<i>echidna_noAbandonedWalletInRegistry</i>	PASS
PROP-12	<i>echidna_registeredWalletsHaveValidBackup</i>	PASS

Note: *echidna_getTriggerableVaultsBatchConsistency* is new in this revision, replacing the prior *checkUpkeep-consistency* check. Two properties (*abandonedWalletsNotInRegistry* and *noAbandonedWalletInRegistry*) test overlapping angles of the same abandonment/registry invariant — intentional redundancy, not a duplication error.

4.5 Gas Report

Function	Min	Avg	Median	Max	Calls
<i>cancelRecovery</i>	23,844	39,077	41,553	58,016	139
<i>getTriggerableVaultsBatch</i>	3,266	4,071	3,266	38,661	273
<i>deposit</i>	23,818	36,917	36,977	39,777	263
<i>getRecoveryConfig</i>	12,944	12,944	12,944	12,944	1,189
<i>getRegisteredWallets</i>	3,331	27,176	39,037	41,988	5
<i>getTimeUntilRecovery</i>	2,997	6,843	7,328	9,719	4
<i>getTotalRegistered</i>	2,530	2,530	2,530	2,530	22
<i>isRecoveryDue</i>	3,037	9,521	9,563	9,563	265
<i>ping</i>	6,400	7,581	6,400	30,264	174
<i>register</i>	29,774	163,224	152,570	209,942	7,288

Function	Min	Avg	Median	Max	Calls
<i>send</i>	24,848	72,291	73,046	75,894	266
<i>triggerRecovery</i>	8,288	8,497	8,288	90,988	43,516
<i>updateBackupAddress</i>	15,681	16,152	15,681	39,913	156
<i>updateInactivityPeriod</i>	13,286	18,427	13,286	37,314	13
<i>withdrawAll</i>	31,129	37,877	39,098	42,184	4

Deployment Cost: 2,726,412 gas. Deployment Size: 12,983 bytes.

Observations relevant to the keeper migration:

- *triggerRecovery*'s wide min/max spread (8,288 to 90,988) reflects the cheap-no-op design: calls against a not-yet-due or already-recovered wallet return almost immediately at floor cost, while an actual recovery execution (transfer + registry removal + state update) costs the higher figure. This directly supports the race-safety design — a losing keeper in a race pays a negligible gas cost, not a wasted full-recovery cost. It also grounds the keeper bot's MAX_CALLS_PER_TX = 120 ceiling: 120 calls at the measured worst-case (90,988 gas) plus a 30% gas-estimation buffer is approximately 14.2M gas — roughly 24% of a 60M block gas limit, comfortable headroom even under network congestion.
- *isRecoveryDue*'s low cost (avg 9,521 gas, max 9,563) confirms it is cheap enough to batch across many database-pre-filter candidates in a single on-chain multicall for pre-submission validation, without materially affecting the economics of the subsequent *triggerRecovery* call. This is the function actually used for that purpose by the Aeternum Labs keeper bot — see Section 3.
- *getTriggerableVaultsBatch*'s low average cost (4,071 gas) makes it cheap to call on-chain as well, but it is not part of the keeper bot's live validation path (see Section 3); its gas profile is included here for completeness, for any other caller that chooses to use it directly.

- register's high average (163,224) and wide spread reflect the first-time storage writes for registry insertion; this is unchanged in kind from the pre-migration contract and is not part of the recovery path.

5. Static Analysis — Slither

Slither v0.11.4 was run against AeternumVault.sol (--solc-remaps "@openzeppelin/contracts/=lib/openzeppelin-contracts/contracts/"). The run returned 27 results across 4 contracts and 101 detectors — down from 34 results across 5 contracts in Revision 2, consistent with the removal of the Chainlink AutomationCompatibleInterface and forwarder surface.

Slither Finding Summary

ID	Detector	Severity	Source	Action
S01	reentrancy-benign in <i>_executeRecovery</i>	Low	Contract	Acknowledged — mitigated (AV-01)
S02	Dangerous timestamp comparisons in <i>getTriggerableVaultsBatch</i> , <i>isRecoveryDue</i> , <i>getTimeUntilRecovery</i> , <i>_executeRecovery</i>	Low	Contract	Acceptable (AV-05)
S03	Inline assembly — <i>getTriggerableVaultsBatch</i> (array length trim) + OpenZeppelin StorageSlot.sol (library)	Informational	Contract/Library	Intentional / third-party
S04	Two Solidity pragma versions in use (^0.8.20 OZ dependencies vs ^0.8.24 project contracts)	Informational	Mixed	Acceptable — pinned compiler resolves conflicts

ID	Detector	Severity	Source	Action
S05	solc-version: OZ ReentrancyGuard.sol / StorageSlot.sol pinned to ^0.8.20, flagged for known compiler issues	Informational	Library	Mitigated (AV-06)
S06	Low-level .call{value}() in <i>withdrawAll</i> , <i>send</i> , <i>cancelRecovery</i> , <i>_executeRecovery</i>	Informational	Contract	Correct modern pattern
S07	naming-convention: MIN_INACTIVITY_PERIOD, MAX_INACTIVITY_PERIOD, MAX_RECOVERY_ATTEMPTS (and IAeternumVault interface equivalents) not mixedCase	Informational	Contract	Intentional — project convention

6. Linting — Solhint

Solhint was run with a custom `.solhint.json` configuration tuned to the project's conventions. `solhint src/AeternumVault.sol` returns 0 errors and 0 warnings.

Rule	Result	Notes
compiler-version	Pass	Pragma pinned to ^0.8.24 across all project files
var-name-mixedcase	Off	s_ prefix and UPPER_SNAKE_CASE for public immutables are conventions
import-path-check	Off	OZ remapping handled in foundry.toml

Rule	Result	Notes
function-max-lines	Off	Some complex functions intentionally exceed 50 lines for readability
gas-strict-inequalities	Off	Non-strict inequalities are intentional throughout
use-natspec @param / @notice	Pass	Full @param, @notice, and @dev coverage across all functions

7. Detailed Findings

AV-01 — Reentrancy-benign in `_executeRecovery`

Severity	Low
Status	Acknowledged — mitigated
Category	Reentrancy
Target	<code>AeternumVault._executeRecovery()</code> (lines 540–592)

Description

On the failure-restore path, state variables (`config.balance` restore, `config.isActive` restore, `s_registeredWallets` push, `s_walletIndexPlusOne`, `s_abandonedBackupAddresses` at line 578) are written after the external ETH transfer call `backupAddress.call{value: amount}()` at line 568. These post-call writes are unavoidable — the contract can only determine a transfer has failed after attempting it. Slither classifies this as reentrancy-benign because `config.balance` is provably zeroed before the call.

Why it is safe / Mitigations

- `nonReentrant` now guards `triggerRecovery()` (the sole external entry point into `_executeRecovery`), blocking all reentrant calls at the outer function level.
- `config.balance` is zeroed BEFORE the external call — even a successful reentrancy attack cannot double-spend.
- After `MAX_RECOVERY_ATTEMPTS` consecutive failures, the vault is permanently abandoned and removed from the registry.

Resolution

The post-call writes on the failure-restore path remain architecturally unavoidable. The slither-disable-next-line annotation is retained with full NatSpec rationale. Unchanged conclusion from Revision 2, migrated to reference *triggerRecovery* in place of *performUpkeep*.

AV-02 — [RESOLVED] External Calls Inside Automation Loop

Severity (prior) **Medium**

Status Resolved — architecture change

Category Denial of Service Risk / Gas Usage

Target (prior) *AeternumVault.performUpkeep()*

Description

In Revision 2, Slither's calls-loop detector flagged the ETH transfer to *backupAddress* inside the *performUpkeep* batch-recovery loop, since a single transaction could iterate over and transfer to multiple distinct backup addresses.

Resolution

The keeper migration eliminates the on-chain loop entirely. *triggerRecovery()* now processes exactly one wallet per call; the contract itself contains no loop over multiple wallets. Batch iteration has moved off-chain into the keeper bot, which submits multiple individual *triggerRecovery* calls per due wallet — typically grouped by an external Multicall3 transaction at the submission layer (see Section 3), outside the audited contract. No function in *AeternumVault.sol* now contains an external call inside a loop. Confirmed by the absence of a calls-loop result in the current Slither run.

AV-03 — [RESOLVED] Missing State-Change Event for *s_checkCursor* Update

Severity (prior) **Informational**

Status Resolved — architecture change

Category Event Emission / Observability

Target (prior) *AeternumVault.performUpkeep()* — *s_checkCursor = nextCursor*

Resolution

The on-chain scan cursor (*s_checkCursor*, *s_lastCursorAdvance*) has been removed entirely. *getTriggerableVaultsBatch* is now a stateless, explicitly-parameterised pagination helper with no persistent on-chain position to track, so there is no cursor state left to require an event.

AV-04 — [RESOLVED] setForwarder() Had No Caller Restriction

Severity (prior) **Informational**

Status Resolved — architecture change

Category Access Control / Operational Risk

Target (prior) *AeternumVault.setForwarder()*

Resolution

The forwarder-gated automation model has been removed. *triggerRecovery()* is now fully permissionless by design, confirmed by *test_triggerRecovery_isCallableByArbitraryAddress*, *test_triggerRecovery_isCallableByBeneficiary*, and *test_triggerRecovery_isCallableByOwnerOfDifferentVault*. This eliminates both the front-run-the-deployment attack surface AV-04 previously described and the single-point-of-failure risk of depending on one automation provider's infrastructure.

AV-05 — Timestamp Comparisons Across View and Recovery Functions

Severity **Low**

Status Acknowledged — acceptable

Category Timestamp Dependence

Target *getTriggerableVaultsBatch*, *isRecoveryDue*, *getTimeUntilRecovery*, *_executeRecovery*

Description

Slither flags *block.timestamp* comparisons against *config.lastActivity* + *config.inactivityPeriod* across four functions — an expansion from one function (*checkUpkeep*) in Revision 2, reflecting that due-wallet logic is now exposed directly as public views (*getTriggerableVaultsBatch*, *isRecoveryDue*) rather than being internal to a single Automation-facing function.

Why it is safe / Mitigations

- Miners/validators can influence *block.timestamp* by at most a few seconds — immaterial against any realistic *MIN_INACTIVITY_PERIOD* (expected in days to years).
- This tolerance directly supports the retained keeper configuration: *KEEPER_POLL_INTERVAL_MS* = 12,000 (12 seconds) is well within the timestamp-manipulation tolerance and does not materially affect recovery timing precision.
- No function relies on sub-second timestamp precision for a security decision (e.g., fund release vs. denial).

Resolution

Acknowledged as acceptable. No code change required.

AV-06 — OpenZeppelin Dependency Uses ^0.8.20 Pragma

Severity	Informational
Status	Acknowledged
Category	Configuration
Target	lib/openzeppelin-contracts/contracts/utils/ReentrancyGuard.sol, StorageSlot.sol

Description

OpenZeppelin's ReentrancyGuard.sol and StorageSlot.sol specify pragma solidity ^0.8.20, which Slither flags as containing known compiler issues: VerbatimInvalidDeduplication, FullInlinerNonExpressionSplitArgumentEvaluationOrder, and MissingSideEffectsOnSelectorAccess.

Why it is safe / Mitigations

- AeternumVault.sol is pinned to pragma solidity ^0.8.24, which resolves all three cited issues.
- The OZ files are third-party dependencies compiled under the project's pinned compiler version and cannot be modified.

Resolution

Acknowledged. No action required. Unchanged from Revision 2.

8. Codebase Maturity Evaluation

Category	Rating	Notes
Arithmetic	Satisfactory	Solidity ^0.8.24 provides native overflow protection. <i>unchecked{}</i> used only in bounded loop counters.

Category	Rating	Notes
Auditing & Logging	Satisfactory	Events cover all state-changing operations — registration, deposits, sends, withdrawals, recovery execution/failure/abandonment/cancellation, activity pings, and config updates. The <i>ForwarderSet</i> event has been removed along with the forwarder-gated automation model.
Access Controls	Satisfactory	<i>onlyRegistered</i> gates all user-facing state-changing functions, correctly admitting both <i>isActive</i> and <i>isAbandoned</i> wallets. No treasury, no admin, no owner role. <i>triggerRecovery()</i> is deliberately permissionless — any address may call it — which is a strengthening of the trustless design compared to the prior forwarder-gated model, removing a single automation provider as a point of failure.
Complexity Management	Satisfactory	Clean separation: user functions, keeper-facing functions (<i>isRecoveryDue</i> , <i>getTriggerableVaultsBatch</i> , <i>triggerRecovery</i>), internal helpers (<i>_executeRecovery</i> , <i>_removeFromRegistry</i>). Interface fully separates type definitions from implementation.
Decentralisation	Satisfactory	No upgradeability. No admin pause. No owner override. No fee collection. The removal of the forwarder/Automation dependency further strengthens decentralisation — recovery no longer depends on a single automation provider's infrastructure remaining available.

Category	Rating	Notes
Documentation	Satisfactory	Comprehensive NatSpec with full @param, @notice, and @dev coverage. Trust model, CEI pattern, and permissionless-keeper rationale documented inline.
Low-Level Manipulation	Satisfactory	Single inline assembly block in <i>getTriggerableVaultsBatch</i> for array length trimming (mstore). Justified for correctness and gas efficiency. Documented and suppressed via Slither comment.
Testing & Verification	Strong	127 total tests (unit, fuzz, and invariant combined), including 6 fuzz functions and 3 Foundry invariants (128,000+ calls each), plus 12 Echidna properties (50,127 calls), and Slither static analysis. 100% line and function coverage on AeternumVault.sol.
Reentrancy	Satisfactory	ReentrancyGuard on <i>triggerRecovery()</i> and all other ETH-transferring functions. CEI enforced throughout. Post-call writes on the failure-restore path are bounded, non-exploitable for fund theft, and classified as reentrancy-benign by Slither.

Category	Rating	Notes
Transaction Ordering	Satisfactory	<i>triggerRecovery()</i> independently re-validates every wallet before acting; stale off-chain keeper data is silently skipped rather than causing incorrect recoveries. Multiple permissionless keepers racing to trigger the same wallet resolve safely — the losing caller's transaction silently returns without reverting or double-transferring funds, whether the competing calls arrive in separate transactions or are batched together via Multicall3. No flash loan or frontrunning attack surface identified for user funds.

9. Known Limitations and Out of Scope

The following items are known limitations of this internal assessment and should be addressed by an external professional audit prior to mainnet deployment:

- The current version supports native ETH only. ERC-20 token support is planned for upcoming phase.
- The frontend application (Next.js) is outside the scope of this contract audit and requires separate security review.
- The Ponder indexer and PostgreSQL database layer used for activity history and the keeper's DB pre-filter are outside the scope of this contract audit and require separate infrastructure and data security review.
- The keeper bot's use of Multicall3 (0xcA11bde05977b3631167028862bE2a173976CA11) for batched transaction submission is outside the scope of this contract-specific audit. Multicall3 is a widely-deployed, immutable, canonical contract used broadly across Ethereum tooling and is not part of AeternumVault.sol; the keeper bot's own gas-estimation and batching logic (apps/keeper of aeternum-infra) has not been separately audited here.
- This assessment was conducted by the contract author. An independent external audit is required before mainnet deployment.

- The Phase 1 keeper bot (Aeternum Labs operated) relies on centralized infrastructure — Railway (indexer & keeper), Neon PostgreSQL — for liveness. A keeper outage does not put funds at risk (recovery is simply delayed, since anyone may call *triggerRecovery*), but the availability of this infrastructure has not been separately audited. Gelato (Phase 2) and Chainlink CRE (Phase 3, operating as a multi-chain orchestrator across independent per-chain deployments) are planned as redundant backup keepers but are not yet live.
- Mainnet gas cost estimates have not been validated under production fork conditions.
- The Echidna fuzzing corpus covers 50,127 calls across 12 properties. Longer runs with higher testLimit values are recommended before external audit engagement.
- The contract does not implement ERC-4337 account abstraction. This is a deliberate design decision, as EIP-7702 hybrid account integration is planned for Phase 2.

10. Tool Versions

Tool	Version	Purpose
Foundry forge	Latest stable (June 2026)	Unit, fuzz, and invariant testing
Slither	0.11.4	Static analysis
Solhint	Latest stable	Solidity linting
Echidna	Latest (Docker — ghcr.io/crytic/echidna)	Property-based fuzzing
Solc	0.8.24	Solidity compiler
OpenZeppelin Contracts	v5.x	ReentrancyGuard dependency

This internal audit report was prepared by Ndubuisi Ugwuja (Aeternum Labs) as a revised pre-audit security assessment. It supersedes the internal audit dated 3rd June 2026. It does not constitute a professional third-party audit. An independent external audit is required before mainnet deployment.

© 2026 Aeternum Labs. All rights reserved.